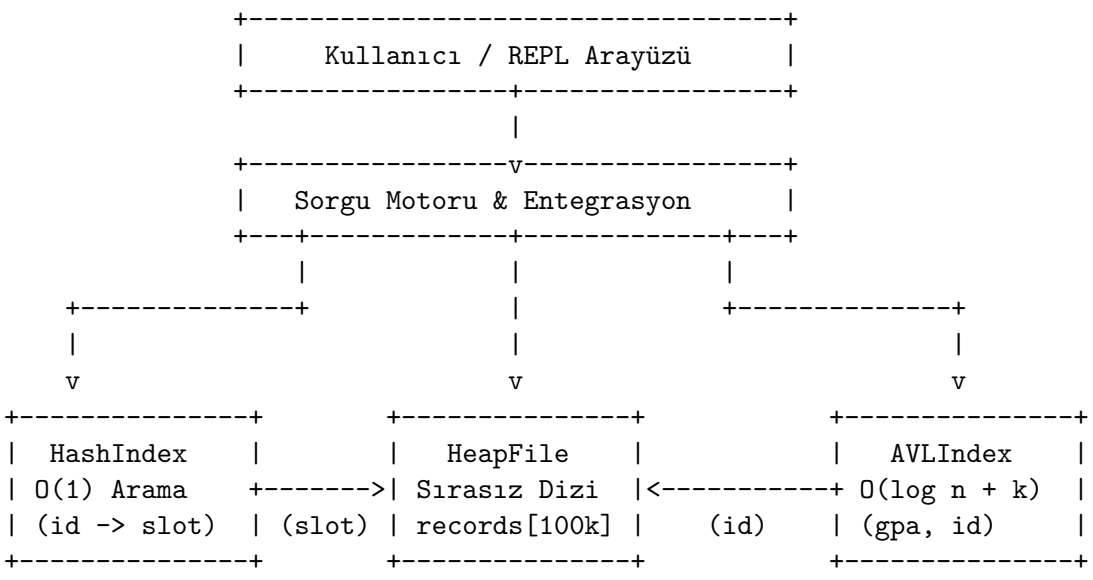
INF224 Proje Ödevi
TinySQL Mini Veritabanı Motoru Teknik Raporu

Muhammed Enis Kartal Yusuf Ziya

Kasım 2026

1 Sistem Mimarisi (Bölüm 13)

TinySQL veritabanı motoru, veri tutarlılığını sağlamak amacıyla katmanlı bir mimariyle tasarlanmıştır. Sistemin merkezinde, tüm fiziksel kayıtları barındıran `HeapFile` yer alır. Verilere hızlı erişim sağlamak için `HashIndex` ve `AVLIndex` yapıları `HeapFile` üzerinde konumlandırılmıştır. Dış dünyadan (kullanıcı veya REPL üzerinden) gelen tüm işlemler yalnızca entegrasyon katmanı olan `Table` modülü üzerinden yürütülür.



- **HeapFile:** Öğrenci kayıtlarının tutulduğu sabit boyutlu statik bir dizidir.
- **HashIndex:** Birincil anahtar olan `id` alanını, kaydın `HeapFile` dizisindeki fiziksel sırasıyla (`slot`) eşleştirir.
- **AVLIndex:** Öğrencilerin not ortalamasını (`gpa`) sıralı tutar. Düğümde sadece `(gpa, id)` ikilisini barındırır.

2 Veri Yapısı Kararları (Bölüm 14)

1. Heap Dosyasında "Swap-with-Last" ile $O(1)$ Silme:
Dizi tabanlı depolamada aradan eleman silindiğinde oluşan boşluğu kapatmak için kaydırma yapmak $O(n)$ maliyet doğurur. Bunun yerine, dizideki son kayıt silinen boşluğa taşınmış (*swap-with-last*) ve işlem $O(1)$ süresine düşürülmüştür. Taşınan bu kaydın Hash tablosundaki konumu `hash_update` fonksiyonuyla yine $O(1)$ sürede güncellenmiştir.

2. Hash Tablosunda Ayrı Zincirleme (Separate Chaining):
Çakışmaları önlemek için Açık Adresleme yerine Ayrı Zincirleme tercih edilmiştir. Sistemde çok fazla `DELETE` işlemi yapılacağı için, açık adreslemenin bıraktığı *tombstone* işaretleri arama performansını düşürecektir. Zincirleme yönteminde ise silinen düğüm doğrudan `free` edildiğinden istikrar korunur. Yük faktörünü optimal tutmak adına tablo kapasitesi 20011 (asal sayı) olarak seçilmiştir.

3. AVL Ağacında Bileşik Anahtar Kullanımı:
Not ortalaması (`gpa`) benzersiz değildir. Bağlı liste açmak yerine **(gpa, id)** bileşik anahtarı kullanılmıştır. Ekleme sırasında GPA'lar eşitse öğrenci ID'si ikincil kriter olarak karşılaştırılarak ağaç dengelenir. Bu sayede tüm operasyonların kesin olarak $O(\log n)$ çalışması sağlanmıştır.

3 Performans Tablosu (Bölüm 15)

Aşağıdaki tablolar, `tools/bench.c` kullanılarak üretilmiş olup teorik Big-O analizlerini deneysel verilerle doğrulamaktadır.

Nokta Sorgusu ve Yük Faktörü

n	Heap Araması $O(n)$ (ns)	Hash Araması $O(1)$ (ns)	Hızlanma
100	23.9	2.0	12x
1.000	130.2	2.6	50x
10.000	1432.9	3.3	434x

Tablo 1: Nokta sorgusu süre karşılaştırması

10.000 kayıtta $\alpha = 0.50$ iken ortalama karşılaştırma teorik (1.250) ve ölçülen (1.258) birbiriyle eşleşmektedir.

Aralık Sorgusu ve Ağaç Doğrulaması

n	Yükseklik	Üst Sınır $\lfloor 1.44 \log_2 n \rfloor$	`avl_verify` Hata	Sınır İçinde
100	8	9	0	Evet
1.000	12	14	0	Evet
10.000	16	19	0	Evet

Tablo 2: AVL Yüksekliği ve Doğrulama

n	k (Bulunan)	AVL $O(\log n + k)$ (μs)	Doğrusal $O(n)$ (μs)	Hızlanma
100	16	0.045	0.065	1.4x
1.000	137	0.301	0.569	1.9x
10.000	1215	3.292	19.561	5.9x

Tablo 3: GPA $\in [3.0, 3.5]$ aralık sorgusu

Aynı Sonucu Veren Farklı Nitelikli Sorgular

`SELECT id = 100500` ile `SELECT dept = "Felsefe"` (tek eşleşme) karşılaştırması:

n	Hash `id` (ns)	Tarama `dept` (ns)	Fark
101	2.4	193.2	79x
1.001	2.5	1834.1	728x
10.001	2.4	17934.2	7363x

Tablo 4: İndeksli vs İndekssiz (Doğrusal) Arama

4 Big-O Analizi (Bölüm 16)

Elde edilen veriler, projenin en başında belirtilen karmaşıklık öngörüleriyle uyuşmaktadır:

- **Ekleme (`INSERT`):** $O(\log n)$. Hash ve Heap kısımları $O(1)$ çalışsa da AVL'ye ekleme operasyonu $O(\log n)$ sürer.
- **Silme (`DELETE`):** $O(\log n)$. Swap-with-last sayesinde Heap $O(1)$, ayrı zincirleme sayesinde Hash ortalama $O(1)$, ağaçtan silme ise $O(\log n)$ olarak tamamlanır.
- **Tekil Arama (`SELECT id`):** Ortalama $O(1)$. Tablo 1'deki sürelerin kayıt sayısından (n) bağımsız şekilde $2.0 \sim 3.3$ ns civarında kalması bunu kanıtlamaktadır.
- **Aralık Arama (`SELECT gpa BETWEEN`):** $O(\log n + k)$. Ağacın yapraklarına inme maliyeti ve bulunan veri (k) sayısıyla orantılı sürede tamamlanmıştır (Tablo 3).

5 Grup Katkı Dağılımı (Bölüm 17)

Proje süreçleri birbirini bloklamadan, senkronize bir şekilde paylaşılmıştır:

- **Muhammed Enis Kartal:** `heap.c`, `hash_index.c`, `avl_index.c`, `table.c`, `query.c` yapılarının ilk sürümlerinin kodlanması. Kod modüllerine ödev formatına uygun Big-O yorum satırlarının eklenmesi, Bölüm 13 ve 14 kapsamındaki veri yapısı teorik raporunun oluşturulması.
- **Yusuf Ziya:** Proje sözleşmesinin (`.h` dosyaları) ve mimari iskeletin oluşturulması, `avl_range` eşleşme sınırlarının düzeltilmesi, `tools/bench.c` ve `tools/gen_csv.c` üzerinden ölçüm testlerinin ve otomatik veri üreticisinin yazılması, bellek sızıntısı (`valgrind` ve `leaks`) denetimlerinin sağlanması.